

EXPORT · THEMING · #1527

# The palette wins the cascade

Every surface on this deck is painted from a token the exported PDF used to ignore.

## THE ONE-LINE CHANGE

# The engine sheet loads first, the palette last.

---

- The order every theme declares
  - Each `themes/*.css` opens with `@import 'lattice';`, which means the engine's rules come first and the theme's own win at equal specificity.
- The order three of four sites already used
  - The Mermaid token reader, `engine.addThemes` and the Studio's composed sheet. Only the export bundle disagreed.
- What that cost
  - 925 declarations across 37 tokens, on all 32 palettes, resolved to the engine's value. A deck looked one way in the Playground and another in the PDF.

---

Render this deck at any palette: what you see is what that palette's author wrote.

# Each palette's own syntax ramp paints now.

```
// Before the flip this panel showed Night Owl's purple and pink on every theme.  
// Now it shows whatever ramp the chosen palette curated for its own code panel.  
  
const ENGINE = readFileSync('dist/lattice.css', 'utf8');  
const PALETTE = themeChain(name).map(readTheme).join('\n');  
  
function deckStylesheet() {  
  // Parent first, engine first: a palette ':root' then beats the base's at  
  // equal specificity – which is what `@import 'lattice';` meant all along.  
  return `${ENGINE}\n${PALETTE}\n/* lattice:palette-end */`;  
}  
  
export const cascade = deckStylesheet(); // one order, every render path
```

Try `cuoio` for warm terracotta and amber, `ardesia` for muted slate and teal.

# The status trio comes from the palette, not the engine.

✓ Done rows take the palette's own pass ink

— Partial rows take its warn ink

amber

○ Open rows keep the neutral ring

✓ An achromatopsia palette paints three grays

⌵ Descoped rows stay struck and muted

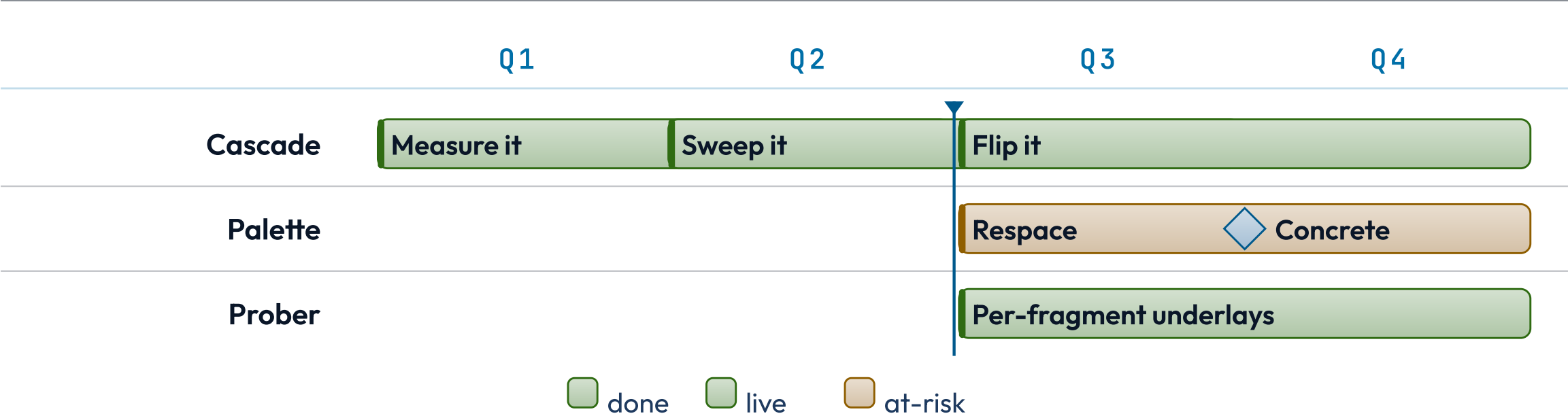
— Three signals, three weights, one palette

On `ally-achromatopsia` these were engine green and amber — one gray to the reader they are for.

2026 Q1 .. 2026 Q4   today Q3

# Plan bars take their status tints from the palette.

*Nine of these tokens used to resolve two ways in one render — the baked SVG from the palette, the CSS around it from the engine.*



# The struck clause sits on a tint of the palette's own red.

lattice-emulator.js:847 · #1527

The deck stylesheet is assembled as ~~the palette then the engine bundle~~ the engine bundle then the palette, so a token the palette declares at `:root` ~~loses to the engine default~~ wins at equal specificity. A palette's syntax ramp, status trio and diagram state family therefore ~~resolve to the engine's value on the export path while the Playground shows the palette's~~ resolve the same way everywhere, and the nine tokens read by both the stylesheet and the Mermaid token map ~~can no longer~~ no longer paint a bar and the rule beside it from one name and two values.

**Why this matters.** The ink and the band behind it both move with the token, so this surface is where a palette's status hue is hardest to read — and where the sweep found the numbers that mattered.

# Same deck, same palette, one cascade

*One order · every render path*

*Preview and export finally agree.*